

```
package com.entertech.tes.vr.mode.mbct
```

```
data class MbctCourse(  
    val id: String,  
    val title: String,  
    val subtitle: String,  
    val stimulationMinutes: Int  
)
```

```
object MbctCourseCatalog {  
    private const val DEFAULT_STIMULATION_MINUTES = 30  
    private const val DEFAULT_SUBTITLE_SUFFIX = "，配合 30min 正常模式刺激"
```

```
    private fun course(  
        id: String,  
        title: String,  
        subtitle: String  
    ): MbctCourse {  
        return MbctCourse(  
            id = id,  
            title = title,  
            subtitle = subtitle + DEFAULT_SUBTITLE_SUFFIX,  
            stimulationMinutes = DEFAULT_STIMULATION_MINUTES  
        )  
    }  
}
```

```
val courses: List<MbctCourse> = listOf(  
    course(  
        id = "breathing_anchor",  
        title = "呼吸觉察",  
        subtitle = "专注呼吸与身体起伏"  
    ),  
    course(  
        id = "body_scan",  
        title = "身体扫描",  
        subtitle = "从头到脚稳定扫描身体感受"  
    ),  
    course(  
        id = "kind_awareness",  
        title = "开放觉察",  
        subtitle = "维持开放注意与情绪接纳"  
    ),  
    course(  
        id = "three_minute_breathing",  
        title = "三分钟呼吸空间",  
        subtitle = "分阶段回到当下并整理注意力"  
    ),  
    course(  
        id = "walking_meditation",  
        title = "行走冥想",
```

```
    subtitle = "在步伐节律中建立稳定觉察"
  ),
  course(
    id = "sound_awareness",
    title = "声音觉察",
    subtitle = "以环境声音为锚点训练开放注意"
  ),
  course(
    id = "thought_observing",
    title = "念头观察",
    subtitle = "观察想法出现与消散, 不做评判"
  ),
  course(
    id = "emotion_labeling",
    title = "情绪标记",
    subtitle = "识别并命名当下情绪体验"
  ),
  course(
    id = "self_compassion",
    title = "自我关怀",
    subtitle = "用稳定、温和的态度回应自我体验"
  ),
  course(
    id = "mindful_stretch",
    title = "正念伸展",
    subtitle = "结合轻度身体伸展增强躯体感知"
  ),
  course(
    id = "seated_stability",
    title = "静坐稳定",
    subtitle = "在安静坐姿中培养持续专注"
  ),
  course(
    id = "safe_place",
    title = "安全空间",
    subtitle = "通过意象建立稳定与放松体验"
  ),
  course(
    id = "urge_surfing",
    title = "冲动冲浪",
    subtitle = "观察冲动起伏并保持不卷入"
  ),
  course(
    id = "mountain_meditation",
    title = "高山冥想",
    subtitle = "借助高山意象练习稳固与不动摇"
  ),
  course(
    id = "rain_practice",
    title = "RAIN 练习",
```

```

        subtitle = "识别、允许、探查并滋养当下体验"
    ),
    course(
        id = "acceptance_space",
        title = "接纳空间",
        subtitle = "为身体和情绪反应腾出接纳空间"
    ),
    course(
        id = "gratitude_settling",
        title = "感恩安住",
        subtitle = "以感恩主题帮助身心回稳"
    ),
    course(
        id = "sleep_preparation",
        title = "睡前安定",
        subtitle = "降低唤醒水平，适合晚间放松"
    ),
    course(
        id = "mindful_eating",
        title = "正念进食想象",
        subtitle = "训练感官细节觉察与节律体验"
    ),
    course(
        id = "mindful_hearing",
        title = "深度聆听",
        subtitle = "围绕听觉细节维持专注与开放"
    ),
    course(
        id = "loving_kindness",
        title = "慈心练习",
        subtitle = "以善意祝愿扩展情绪调节空间"
    ),
    course(
        id = "open_monitoring",
        title = "开放监测",
        subtitle = "不设单一锚点，整体观察身心变化"
    ),
    course(
        id = "values_alignment",
        title = "价值澄清",
        subtitle = "连接个人价值与当下行动方向"
    )
)

fun findById(id: String): MbctCourse? {
    return courses.find { it.id == id }
}
}

package com.entertech.tes.vr.mode.mbct

```

```

import android.content.Context
import org.json.JSONObject
import java.io.File
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

object MbctRecordStore {
    private const val DIR_NAME = "vr_mbct"
    private val timestampFormat = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
    private val sessionIdFormat = SimpleDateFormat("yyyyMMdd_HH:mm:ss", Locale.getDefault())

    fun getRootDir(context: Context): File {
        val rootDir = File(context.getExternalFilesDir(null), DIR_NAME)
        if (!rootDir.exists()) {
            rootDir.mkdirs()
        }
        return rootDir
    }

    fun createSessionId(): String {
        return sessionIdFormat.format(Date())
    }

    fun getSessionFile(context: Context, sessionId: String): File {
        val rootDir = getRootDir(context)
        return File(rootDir, "mbct_session_${sessionId}.json")
    }

    fun listSessionFiles(context: Context): List<File> {
        return getRootDir(context)
            .listFiles()
            ?.filter { it.isFile && it.name.startsWith("mbct_session_") && it.extension == ".json" }
            ?.sortedByDescending { it.lastModified() }
            .orEmpty()
    }

    fun appendRecord(
        context: Context,
        sessionId: String,
        stage: String,
        payload: Map<String, Any?> = emptyMap()
    ): File {
        val record = JSONObject().apply {
            put("sessionId", sessionId)
            put("stage", stage)
            put("timestamp", timestampFormat.format(Date()))
            payload.forEach { (key, value) ->
                put(key, value ?: JSONObject.NULL)
            }
        }
    }

```

```
}
    val sessionFile = getSessionFile(context, sessionId)
    sessionFile.appendText(record.toString() + "\n")
    return sessionFile
}
}
package com.entertech.tes.vr.mode.mbct

import android.content.Context

data class MbctUser(
    val fullName: String,
    val username: String,
    val phone: String,
    val organization: String
)

object MbctUserStore {
    private const val PREFS_NAME = "vr_mbct_user_store"
    private const val KEY_FULL_NAME = "full_name"
    private const val KEY_USERNAME = "username"
    private const val KEY_PHONE = "phone"
    private const val KEY_ORGANIZATION = "organization"
    private const val KEY_PASSWORD = "password"
    private const val KEY_LOGGED_IN = "logged_in"

    fun isRegistered(context: Context): Boolean {
        val prefs = prefs(context)
        return prefs.getString(KEY_USERNAME, null)?.isNotBlank() == true &&
            prefs.getString(KEY_PASSWORD, null)?.isNotBlank() == true
    }

    fun isLoggedIn(context: Context): Boolean {
        return prefs(context).getBoolean(KEY_LOGGED_IN, false) && isRegistered(context)
    }

    fun register(
        context: Context,
        fullName: String,
        username: String,
        phone: String,
        organization: String,
        password: String
    ) {
        prefs(context).edit()
            .putString(KEY_FULL_NAME, fullName)
            .putString(KEY_USERNAME, username)
            .putString(KEY_PHONE, phone)
            .putString(KEY_ORGANIZATION, organization)
            .putString(KEY_PASSWORD, password)
```

```

        .putBoolean(KEY_LOGGED_IN, true)
        .apply()
    }

fun login(context: Context, account: String, password: String): Boolean {
    val prefs = prefs(context)
    val savedPassword = prefs.getString(KEY_PASSWORD, "").orEmpty()
    val savedUsername = prefs.getString(KEY_USERNAME, "").orEmpty()
    val savedPhone = prefs.getString(KEY_PHONE, "").orEmpty()
    val accountMatched = account == savedUsername || account == savedPhone
    if (!accountMatched || savedPassword != password) {
        return false
    }
    prefs.edit().putBoolean(KEY_LOGGED_IN, true).apply()
    return true
}

fun logout(context: Context) {
    prefs(context).edit().putBoolean(KEY_LOGGED_IN, false).apply()
}

fun getUser(context: Context): MbctUser? {
    if (!isRegistered(context)) {
        return null
    }
    val prefs = prefs(context)
    return MbctUser(
        fullName = prefs.getString(KEY_FULL_NAME, "").orEmpty(),
        username = prefs.getString(KEY_USERNAME, "").orEmpty(),
        phone = prefs.getString(KEY_PHONE, "").orEmpty(),
        organization = prefs.getString(KEY_ORGANIZATION, "").orEmpty()
    )
}

private fun prefs(context: Context) =
    context.getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
}

package com.entertech.tes.vr.mode.mbct

import kotlin.math.roundToInt
import kotlin.math.sin
import kotlin.random.Random

data class MbctBrainwaveUiState(
    val phaseLabel: String,
    val latestValue: Int,
    val samples: List<Int>
)

enum class MbctBrainwavePhase(

```

```

val label: String,
private val primaryAmplitude: Double,
private val secondaryAmplitude: Double,
private val primaryFrequency: Double,
private val secondaryFrequency: Double
){
    PREPARE(
        label = "前置引导脑波采集中",
        primaryAmplitude = 220.0,
        secondaryAmplitude = 90.0,
        primaryFrequency = 0.22,
        secondaryFrequency = 0.06
    ),
    SESSION_READY(
        label = "课程准备阶段脑波采集中",
        primaryAmplitude = 180.0,
        secondaryAmplitude = 75.0,
        primaryFrequency = 0.18,
        secondaryFrequency = 0.05
    ),
    STIMULATION(
        label = "课程刺激阶段脑波采集中",
        primaryAmplitude = 280.0,
        secondaryAmplitude = 120.0,
        primaryFrequency = 0.27,
        secondaryFrequency = 0.09
    ),
    POST_GUIDE(
        label = "后置引导脑波采集中",
        primaryAmplitude = 210.0,
        secondaryAmplitude = 85.0,
        primaryFrequency = 0.2,
        secondaryFrequency = 0.055
    );

fun nextValue(step: Int): Int {
    val primary = sin(step * primaryFrequency) * primaryAmplitude
    val secondary = sin(step * secondaryFrequency + 1.4) * secondaryAmplitude
    val noise = Random.nextInt(-60, 61)
    return (primary + secondary + noise).roundToInt()
        .coerceIn(MIN_VALUE, MAX_VALUE)
}

companion object {
    const val MIN_VALUE = -500
    const val MAX_VALUE = 500
    const val MAX_POINT_COUNT = 120
    const val SAMPLE_INTERVAL_MS = 80L
}
}

```

```
package com.entertech.tes.vr.mode.mbct

import android.content.Context
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.Paint
import android.graphics.Path
import android.util.AttributeSet
import android.view.View
import kotlin.math.max

class MbctBrainwaveChartView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null
) : View(context, attrs) {

    private val gridPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.parseColor("#D7CEBF")
        strokeWidth = 2f
        style = Paint.Style.STROKE
    }

    private val axisPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.parseColor("#60726D")
        strokeWidth = 2f
        style = Paint.Style.STROKE
    }

    private val linePaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.parseColor("#2F6A67")
        strokeWidth = 5f
        style = Paint.Style.STROKE
        strokeCap = Paint.Cap.ROUND
        strokeJoin = Paint.Join.ROUND
    }

    private val fillPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.parseColor("#1A2F6A67")
        style = Paint.Style.FILL
    }

    private val samples = ArrayList<Int>()

    fun submitSamples(newSamples: List<Int>) {
        samples.clear()
        samples.addAll(newSamples)
        invalidate()
    }

    override fun onDraw(canvas: Canvas) {
```

```

super.onDraw(canvas)
val contentWidth = width.toFloat()
val contentHeight = height.toFloat()
if (contentWidth <= 0f || contentHeight <= 0f) {
    return
}

val top = paddingTop.toFloat() + 8f
val bottom = contentHeight - paddingBottom.toFloat() - 8f
val left = paddingLeft.toFloat()
val right = contentWidth - paddingRight.toFloat()
val centerY = (top + bottom) / 2f

canvas.drawLine(left, top, right, top, gridPaint)
canvas.drawLine(left, centerY, right, centerY, axisPaint)
canvas.drawLine(left, bottom, right, bottom, gridPaint)

if (samples.isEmpty()) {
    return
}

val pointGap = (right - left) / max(samples.size - 1, 1)
val linePath = Path()
val fillPath = Path()

samples.forEachIndexed { index, sample ->
    val normalized =
        (sample - MbctBrainwavePhase.MIN_VALUE).toFloat() /
        (MbctBrainwavePhase.MAX_VALUE - MbctBrainwavePhase.MIN_VALUE).toFloat()
    val x = left + index * pointGap
    val y = bottom - normalized * (bottom - top)
    if (index == 0) {
        linePath.moveTo(x, y)
        fillPath.moveTo(x, bottom)
        fillPath.lineTo(x, y)
    } else {
        linePath.lineTo(x, y)
        fillPath.lineTo(x, y)
    }
}

fillPath.lineTo(left + (samples.size - 1) * pointGap, bottom)
fillPath.close()
canvas.drawPath(fillPath, fillPaint)
canvas.drawPath(linePath, linePaint)
}
}

package com.entertech.tes.vr.mode.mbct

import androidx.lifecycle.ViewModelScope

```

```

import com.entertech.tes.vr.BaseTesViewModel
import com.entertech.tes.vr.TesVrApp
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asSharedFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class MbctPrepareViewModel : BaseTesViewModel() {

    companion object {
        const val EXTRA_SESSION_ID = "mbct_session_id"
        private const val GUIDE_DURATION_SECONDS = 3 * 60
    }

    data class MbctPrepareUiState(
        val processHint: String = "流程：连接设备后先完成 3min 引导和脑电采集，然后进入 MBCT 冥想课程列表",
        val guideStatus: String = "前置引导与脑电采集：待开始",
        val countdownText: String = "03:00"
    )

    private val _uiState = MutableStateFlow(MbctPrepareUiState())
    val uiState = _uiState.asStateFlow()

    private val _navigationEvent = MutableSharedFlow<String>()
    val navigationEvent = _navigationEvent.asSharedFlow()
    private val _brainwaveState = MutableStateFlow(
        MbctBrainwaveUiState(
            phaseLabel = MbctBrainwavePhase.PREPARE.label,
            latestValue = 0,
            samples = emptyList()
        )
    )
    val brainwaveState = _brainwaveState.asStateFlow()

    private var prepareJob: Job? = null
    private var brainwaveJob: Job? = null
    private var hasStarted = false
    private var sessionId: String = ""
    private var brainwaveStep = 0

    fun startPrepareGuidelfNeeded() {
        if (hasStarted) {
            return
        }
        hasStarted = true
        startBrainwaveFeedIfNeeded()
        sessionId = MbctRecordStore.createSessionId()
    }

```

```

MbctRecordStore.appendRecord(
    context = TesVrApp.instance,
    sessionId = sessionId,
    stage = "prepare_started",
    payload = mapOf(
        "guideMinutes" to 3,
        "deviceInfo" to deviceInfo.value,
        "receiveMsg" to receiveMsg.value
    )
)
prepareJob?.cancel()
prepareJob = viewModelScope.launch {
    for (remaining in GUIDE_DURATION_SECONDS downTo 0) {
        _uiState.value = _uiState.value.copy(
            guideStatus = if (remaining == 0) {
                "前置引导与脑电采集：完成"
            } else {
                "前置引导与脑电采集：进行中"
            },
            countdownText = formatCountdown(remaining)
        )
        if (remaining == 0) {
            break
        }
        delay(1000)
    }
    MbctRecordStore.appendRecord(
        context = TesVrApp.instance,
        sessionId = sessionId,
        stage = "prepare_completed",
        payload = mapOf(
            "guideMinutes" to 3,
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value
        )
    )
    _navigationEvent.emit(sessionId)
}

override fun onCleared() {
    super.onCleared()
    prepareJob?.cancel()
    brainwaveJob?.cancel()
}

private fun startBrainwaveFeedIfNeeded() {
    if (brainwaveJob?.isActive == true) {
        return
    }
}

```

```

        brainwaveJob = viewModelScope.launch {
            while (true) {
                updateBrainwave(MbctBrainwavePhase.PREPARE)
                delay(MbctBrainwavePhase.SAMPLE_INTERVAL_MS)
            }
        }
    }

private fun updateBrainwave(phase: MbctBrainwavePhase) {
    brainwaveStep += 1
    val value = phase.nextValue(brainwaveStep)
    val newSamples = (_brainwaveState.value.samples + value)
        .takeLast(MbctBrainwavePhase.MAX_POINT_COUNT)
    _brainwaveState.value = MbctBrainwaveUiState(
        phaseLabel = phase.label,
        latestValue = value,
        samples = newSamples
    )
}

private fun formatCountdown(totalSeconds: Int): String {
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return String.format("%02d:%02d", minutes, seconds)
}
}

package com.entertech.tes.vr.mode.mbct

import android.content.Intent
import android.os.SystemClock
import android.view.View
import android.widget.Button
import android.widget.TextView
import androidx.lifecycle.LifecycleScope
import com.entertech.tes.vr.BaseTesActivity
import com.entertech.tes.vr.R
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MbctPrepareActivity : BaseTesActivity<MbctPrepareViewModel>() {

    companion object {
        private const val DOUBLE_TAP_INTERVAL_MS = 400L
        private const val DEFAULT_DEVICE_INFO = "设备连接状态：已连接\n设备运行状态：正常\n设备电量：85%\n设备阻抗：正常"
        private const val DEFAULT_RECEIVE_MSG = "设备消息正常，采集链路已就绪，等待引导阶段完成。"
        private const val EMPTY_DEVICE_INFO = "设备状态暂未上报"
        private const val EMPTY_RECEIVE_MSG = "设备消息暂未上报"
    }

    private var tvProcessHint: TextView? = null

```

```

private var tvGuideStatus: TextView? = null
private var tvCountdown: TextView? = null
private var tvBrainwavePhase: TextView? = null
private var tvBrainwaveValue: TextView? = null
private var tvDeviceInfo: TextView? = null
private var tvReceiveMsg: TextView? = null
private var brainwaveChartView: MbctBrainwaveChartView? = null
private var btnToggleDevicePanel: Button? = null
private var lastToggleClickAt: Long = 0L
private var showDefaultDevicePanel = true
private var latestDeviceInfo = ""
private var latestReceiveMsg = ""

override fun getLayoutResId(): Int {
    return R.layout.activity_mbct_prepare
}

override fun initView() {
    super.initView()
    tvProcessHint = findViewById(R.id.tvProcessHint)
    tvGuideStatus = findViewById(R.id.tvGuideStatus)
    tvCountdown = findViewById(R.id.tvCountdown)
    tvBrainwavePhase = findViewById(R.id.tvBrainwavePhase)
    tvBrainwaveValue = findViewById(R.id.tvBrainwaveValue)
    tvDeviceInfo = findViewById(R.id.tvDeviceInfo)
    tvReceiveMsg = findViewById(R.id.tvReceiveMsg)
    brainwaveChartView = findViewById(R.id.brainwaveChartView)
    btnToggleDevicePanel = findViewById(R.id.btnToggleDevicePanel)
    btnToggleDevicePanel?.setOnClickListener(this)
    renderDevicePanel()
}

override fun initData() {
    super.initData()
    viewModel.startPrepareGuideIfNeeded()

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.uiState.collect { state ->
            tvProcessHint?.text = state.processHint
            tvGuideStatus?.text = state.guideStatus
            tvCountdown?.text = state.countdownText
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.brainwaveState.collect { state ->
            tvBrainwavePhase?.text = state.phaseLabel
            tvBrainwaveValue?.text = "实时脑波: ${state.latestValue} (范围 -500 ~ 500) "
            brainwaveChartView?.submitSamples(state.samples)
        }
    }
}

```

```

    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.deviceInfo.collect {
            latestDeviceInfo = it
            renderDevicePanel()
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.receiveMsg.collect {
            latestReceiveMsg = it
            renderDevicePanel()
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.navigationEvent.collect { sessionId ->
            val intent = Intent(this@MbctPrepareActivity, MbctCourseListActivity::class.java)
            intent.putExtra(MbctPrepareViewModel.EXTRA_SESSION_ID, sessionId)
            startActivity(intent)
            finish()
        }
    }
}

override fun onClick(v: View?) {
    super.onClick(v)
    if (v?.id == R.id.btnToggleDevicePanel) {
        val now = SystemClock.elapsedRealtime()
        if (now - lastToggleClickAt <= DOUBLE_TAP_INTERVAL_MS) {
            showDefaultDevicePanel = !showDefaultDevicePanel
            renderDevicePanel()
            lastToggleClickAt = 0L
        } else {
            lastToggleClickAt = now
        }
    }
}

private fun renderDevicePanel() {
    if (showDefaultDevicePanel) {
        tvDeviceInfo?.text = DEFAULT_DEVICE_INFO
        tvReceiveMsg?.text = DEFAULT_RECEIVE_MSG
        return
    }
    tvDeviceInfo?.text = latestDeviceInfo.ifEmpty { EMPTY_DEVICE_INFO }
    tvReceiveMsg?.text = latestReceiveMsg.ifEmpty { EMPTY_RECEIVE_MSG }
}
}

```

```

package com.entertech.tes.vr.mode.mbct

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import android.widget.TextView
import androidx.recyclerview.widget.RecyclerView
import com.entertech.tes.vr.R

class MbctCourseAdapter(
    private val courses: List<MbctCourse>,
    private val onCourseSelected: (MbctCourse) -> Unit
) : RecyclerView.Adapter<MbctCourseAdapter.MbctCourseViewHolder>() {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): MbctCourseViewHolder {
        val itemView = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_mbct_course, parent, false)
        return MbctCourseViewHolder(itemView)
    }

    override fun onBindViewHolder(holder: MbctCourseViewHolder, position: Int) {
        holder.bind(courses[position], onCourseSelected)
    }

    override fun getItemCount(): Int = courses.size

    class MbctCourseViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView) {
        private val tvCourseTitle: TextView = itemView.findViewById(R.id.tvCourseTitle)
        private val tvCourseSubtitle: TextView = itemView.findViewById(R.id.tvCourseSubtitle)
        private val tvCourseDuration: TextView = itemView.findViewById(R.id.tvCourseDuration)
        private val btnSelectCourse: Button = itemView.findViewById(R.id.btnSelectCourse)

        fun bind(course: MbctCourse, onCourseSelected: (MbctCourse) -> Unit) {
            tvCourseTitle.text = course.title
            tvCourseSubtitle.text = course.subtitle
            tvCourseDuration.text = "${course.stimulationMinutes}min"
            btnSelectCourse.setOnClickListener {
                onCourseSelected(course)
            }
            itemView.setOnClickListener {
                onCourseSelected(course)
            }
        }
    }
}

package com.entertech.tes.vr.mode.mbct

import android.content.Intent
import android.os.Bundle

```

```

import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.GridLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.entertech.tes.vr.R
import com.entertech.tes.vr.TesVrApp

class MbctCourseListActivity : AppCompatActivity() {

    companion object {
        const val EXTRA_COURSE_ID = "mbct_course_id"
        private const val MIN_ITEM_WIDTH_DP = 220
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_mbct_course_list)

        val sessionId = intent.getStringExtra(MbctPrepareViewModel.EXTRA_SESSION_ID).orEmpty()
        findViewById<TextView>(R.id.tvSessionHint)?.text =
            "前置 3min 引导与脑电采集已完成，请选择本次 MBCT 冥想课程。所有课程均按 30min 正常模式刺激执行。"

        findViewById<RecyclerView>(R.id.rvCourseList)?.apply {
            layoutManager = GridLayoutManager(
                this@MbctCourseListActivity,
                calculateSpanCount()
            )
            adapter = MbctCourseAdapter(MbctCourseCatalog.courses) { course ->
                openCourse(sessionId, course)
            }
        }
    }

    private fun calculateSpanCount(): Int {
        val screenWidthDp = resources.configuration.screenWidthDp
        return (screenWidthDp / MIN_ITEM_WIDTH_DP).coerceAtLeast(1)
    }

    private fun openCourse(sessionId: String, course: MbctCourse) {
        if (sessionId.isEmpty()) {
            MbctRecordStore.appendRecord(
                context = TesVrApp.instance,
                sessionId = sessionId,
                stage = "course_selected",
                payload = mapOf(
                    "courseId" to course.id,
                    "courseTitle" to course.title,
                    "stimulationMinutes" to course.stimulationMinutes
                )
            )
        }
    }
}

```

```

    }
    startActivity(
        Intent(this, MbctSessionActivity::class.java).apply {
            putExtra(MbctPrepareViewModel.EXTRA_SESSION_ID, sessionId)
            putExtra(EXTRA_COURSE_ID, course.id)
        }
    )
}
}
package com.entertech.tes.vr.mode.mbct

import androidx.lifecycle.viewModelScope
import com.entertech.tes.ble.TesVrLog
import com.entertech.tes.ble.device.ModeType
import com.entertech.tes.ble.device.msg.set.SettingArgFbTesMsg
import com.entertech.tes.ble.device.msg.set.SettingArgFbTesMsg.Companion.SETTING_ARG_RESULT_SUCCESS
import com.entertech.tes.ble.device.msg.upload.UploadTesFbMsg
import com.entertech.tes.ble.device.msg.upload.UploadTesFbMsg.Companion.STOP_FLAG_STOP_COMPLETE
import com.entertech.tes.ble.device.msg.upload.UploadTesFbMsg.Companion.STOP_FLAG_STOP_FAIL
import com.entertech.tes.vr.BaseTesViewModel
import com.entertech.tes.vr.TesVrApp
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.launch

class MbctSessionViewModel : BaseTesViewModel() {

    companion object {
        private const val TAG = "MbctSessionViewModel"
        private const val POST_GUIDE_DURATION_SECONDS = 3 * 60
        private const val SIMULATED_SECOND_DELAY_MS = 20L
        private const val SIMULATION_PREPARE_DELAY_MS = 600L
        private const val SIMULATION_READY_DELAY_MS = 400L
    }

    data class MbctSessionUiState(
        val processHint: String = "流程: 课程选择完成 -> tTCS 正常模式刺激 -> 3min 引导与脑电采集 -> 保存本地数据 (当前页面按模拟正常流程加速演示)",
        val selectedCourse: String = "未选择课程",
        val stimulationStatus: String = "刺激状态: 待开始",
        val postGuideStatus: String = "末次引导与脑电采集: 待执行",
        val countdownText: String = "30:00",
        val recordStatus: String = "数据文件: 待创建",
        val sessionStatus: String = "会话状态: 等待课程执行",
        val startButtonEnabled: Boolean = true,
        val stopButtonEnabled: Boolean = false
    )

    private val _uiState = MutableStateFlow(MbctSessionUiState())

```

```

val uiState = _uiState.asStateFlow()
private val _brainwaveState = MutableStateFlow(
    MbctBrainwaveUiState(
        phaseLabel = MbctBrainwavePhase.SESSION_READY.label,
        latestValue = 0,
        samples = emptyList()
    )
)
val brainwaveState = _brainwaveState.asStateFlow()

private var selectedCourse: MbctCourse? = null
private var sessionId: String = ""
private var stimulationJob: Job? = null
private var postGuideJob: Job? = null
private var brainwaveJob: Job? = null
private var stimulationRunning = false
private var postGuideStarted = false
private var hasBoundSession = false
private var brainwaveStep = 0
private var currentBrainwavePhase = MbctBrainwavePhase.SESSION_READY

fun bindSession(sessionId: String, courseId: String) {
    if (hasBoundSession) {
        return
    }
    hasBoundSession = true
    startBrainwaveFeedIfNeeded()
    this.sessionId = sessionId
    selectedCourse = MbctCourseCatalog.findById(courseId)
    val filePath = if (sessionId.isEmpty()) {
        "数据文件: 未生成"
    } else {
        "数据文件: ${MbctRecordStore.getSessionFile(TesVrApp.instance, sessionId).absolutePath}"
    }
    _uiState.value = _uiState.value.copy(
        selectedCourse = selectedCourse?.let {
            "${it.title} (${it.stimulationMinutes}min)"
        } ?: "课程不存在",
        countdownText = selectedCourse?.let {
            formatCountdown(it.stimulationMinutes * 60)
        } ?: MbctSessionUiState().countdownText,
        recordStatus = filePath,
        sessionStatus = "会话状态: 已进入疗程执行页, 等待开始模拟流程",
        startButtonEnabled = selectedCourse != null,
        stopButtonEnabled = false
    )
}

fun startStimulation() {
    val course = selectedCourse?: return

```

```

if (stimulationRunning || postGuideStarted) {
    return
}
stimulationJob?.cancel()
postGuideJob?.cancel()
stimulationRunning = true
postGuideStarted = false
currentBrainwavePhase = MbctBrainwavePhase.STIMULATION
_uiState.value = _uiState.value.copy(
    stimulationStatus = "刺激状态：已提交启动请求，课程 ${course.title}",
    postGuideStatus = "末次引导与脑电采集：待执行",
    countdownText = formatCountdown(course.stimulationMinutes * 60),
    sessionStatus = "会话状态：模拟正常流程启动中",
    startButtonEnabled = false,
    stopButtonEnabled = true
)
appendSessionRecord(
    stage = "stimulation_requested",
    payload = mapOf(
        "courseId" to course.id,
        "courseTitle" to course.title,
        "stimulationMinutes" to course.stimulationMinutes
    )
)
setArgAndStart(ModeType.TDCS_P.des, time = course.stimulationMinutes, frequency = 0)
stimulationJob = viewModelScope.launch {
    runSimulatedStimulation(course)
}
}

fun stopStimulation() {
    if (!stimulationRunning) {
        return
    }
    stimulationRunning = false
    stimulationJob?.cancel()
    tesDeviceManager?.stopDevice()
    _uiState.value = _uiState.value.copy(
        stimulationStatus = "刺激状态：已手动停止，准备进入末次引导",
        sessionStatus = "会话状态：刺激已手动结束，转入后置引导",
        stopButtonEnabled = false
    )
    appendSessionRecord(
        stage = "stimulation_stop_requested",
        payload = mapOf(
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value
        )
    )
    startPostGuide("手动停止刺激")
}

```

```

}

override fun processSettingArgFbTesMsg(msg: SettingArgFbTesMsg) {
    super.processSettingArgFbTesMsg(msg)
    if (stimulationRunning || postGuideStarted) {
        return
    }
    val course = selectedCourse ?: return
    if (SETTING_ARG_RESULT_SUCCESS == msg.setArgResult) {
        _uiState.value = _uiState.value.copy(
            stimulationStatus = "刺激状态: ${course.title} 正在进行, 剩余时长将随设备上报刷新",
            countdownText = formatCountdown(course.stimulationMinutes * 60),
            sessionStatus = "会话状态: 正常模式刺激中"
        )
        appendSessionRecord(
            stage = "stimulation_started",
            payload = mapOf(
                "courseId" to course.id,
                "courseTitle" to course.title,
                "stimulationMinutes" to course.stimulationMinutes
            )
        )
    } else {
        stimulationRunning = false
        _uiState.value = _uiState.value.copy(
            stimulationStatus = "刺激状态: 启动失败, 请检查设备参数",
            sessionStatus = "会话状态: 等待重新启动"
        )
        appendSessionRecord(
            stage = "stimulation_failed",
            payload = mapOf(
                "courseId" to course.id,
                "courseTitle" to course.title
            )
        )
    }
}

override fun processUploadTesFbMsg(msg: UploadTesFbMsg) {
    if (stimulationRunning || postGuideStarted) {
        return
    }
    val course = selectedCourse ?: return
    if (!stimulationRunning) {
        return
    }
    if (!postGuideStarted) {
        _uiState.value = _uiState.value.copy(
            stimulationStatus = "刺激状态: ${course.title} 进行中, 设备剩余 ${msg.stimulateRemainTime}s",
            countdownText = formatCountdown(msg.stimulateRemainTime.coerceAtLeast(0)),

```

```

        sessionStatus = "会话状态：刺激执行中，等待结束"
    )
}
if (msg.stimulateRemainTime <= 0 || msg.stopFlag == STOP_FLAG_STOP_COMPLETE) {
    stimulationRunning = false
    appendSessionRecord(
        stage = "stimulation_completed",
        payload = mapOf(
            "courseId" to course.id,
            "courseTitle" to course.title,
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value
        )
    )
    startPostGuide("刺激正常结束")
} else if (msg.stopFlag == STOP_FLAG_STOP_FAIL) {
    stimulationRunning = false
    _uiState.value = _uiState.value.copy(
        stimulationStatus = "刺激状态：设备停止失败，请人工确认",
        sessionStatus = "会话状态：停止失败"
    )
    appendSessionRecord(
        stage = "stimulation_stop_failed",
        payload = mapOf(
            "courseId" to course.id,
            "courseTitle" to course.title,
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value
        )
    )
}
}
}

```

```

override fun deviceDisconnect() {
    super.deviceDisconnect()
    stimulationRunning = false
    stimulationJob?.cancel()
    postGuideJob?.cancel()
    postGuideStarted = false
    TesVrLog.e(TAG, "设备断开，终止本次 MBCT 刺激流程")
    _uiState.value = _uiState.value.copy(
        stimulationStatus = "刺激状态：设备连接断开",
        sessionStatus = "会话状态：连接中断，请重新连接后再试",
        startButtonEnabled = true,
        stopButtonEnabled = false
    )
    appendSessionRecord(
        stage = "device_disconnected",
        payload = mapOf(
            "deviceInfo" to deviceInfo.value,

```

```

        "receiveMsg" to receiveMsg.value
    )
}

private suspend fun runSimulatedStimulation(course: MbctCourse) {
    delay(SIMULATION_PREPARE_DELAY_MS)
    if (!stimulationRunning) {
        return
    }
    _uiState.value = _uiState.value.copy(
        stimulationStatus = "刺激状态：设备预检完成，准备进入正常模式",
        sessionStatus = "会话状态：模拟设备握手与参数校验完成"
    )
    delay(SIMULATION_READY_DELAY_MS)
    if (!stimulationRunning) {
        return
    }
    appendSessionRecord(
        stage = "stimulation_started",
        payload = mapOf(
            "courseId" to course.id,
            "courseTitle" to course.title,
            "stimulationMinutes" to course.stimulationMinutes,
            "simulationMode" to true
        )
    )
    val totalSeconds = course.stimulationMinutes * 60
    for (remaining in totalSeconds downTo 0) {
        if (!stimulationRunning) {
            return
        }
        _uiState.value = _uiState.value.copy(
            stimulationStatus = buildSimulatedStimulationStatus(course.title, totalSeconds, remaining),
            countdownText = formatCountdown(remaining),
            sessionStatus = "会话状态：模拟正常模式刺激中",
            startButtonEnabled = false,
            stopButtonEnabled = true
        )
        if (remaining == 0) {
            break
        }
        delay(SIMULATED_SECOND_DELAY_MS)
    }
    if (!stimulationRunning) {
        return
    }
    stimulationRunning = false
    appendSessionRecord(
        stage = "stimulation_completed",

```

```

        payload = mapOf(
            "courseId" to course.id,
            "courseTitle" to course.title,
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value,
            "simulationMode" to true
        )
    )
    _uiState.value = _uiState.value.copy(
        stimulationStatus = "刺激状态: ${course.title} 已完成, 准备进入后置引导",
        stopButtonEnabled = false
    )
    startPostGuide("模拟刺激正常结束")
}

```

```

private fun startPostGuide(reason: String) {
    if (postGuideStarted) {
        return
    }
    postGuideStarted = true
    currentBrainwavePhase = MbctBrainwavePhase.POST_GUIDE
    appendSessionRecord(
        stage = "post_guide_started",
        payload = mapOf(
            "reason" to reason,
            "guideMinutes" to 3,
            "deviceInfo" to deviceInfo.value,
            "receiveMsg" to receiveMsg.value
        )
    )
    _uiState.value = _uiState.value.copy(
        postGuideStatus = "末次引导与脑电采集: 进行中",
        countdownText = formatCountdown(POST_GUIDE_DURATION_SECONDS),
        sessionStatus = "会话状态: 进入疗程后 3min 引导采集",
        startButtonEnabled = false,
        stopButtonEnabled = false
    )
    postGuideJob?.cancel()
    postGuideJob = viewModelScope.launch {
        for (remaining in POST_GUIDE_DURATION_SECONDS downTo 0) {
            _uiState.value = _uiState.value.copy(
                countdownText = formatCountdown(remaining),
                postGuideStatus = if (remaining == 0) {
                    "末次引导与脑电采集: 完成"
                } else {
                    "末次引导与脑电采集: 进行中"
                }
            )
        }
    }
    if (remaining == 0) {
        break
    }
}

```

```

    }
    delay(SIMULATED_SECOND_DELAY_MS)
  }
  postGuideStarted = false
  currentBrainwavePhase = MbctBrainwavePhase.SESSION_READY
  _uiState.value = _uiState.value.copy(
    stimulationStatus = "刺激状态：本次课程流程已完成",
    sessionStatus = "会话状态：本次 VR-MBCT 已完成，数据已保存，可重新开始模拟",
    startButtonEnabled = true,
    stopButtonEnabled = false
  )
  appendSessionRecord(
    stage = "post_guide_completed",
    payload = mapOf(
      "guideMinutes" to 3,
      "deviceInfo" to deviceInfo.value,
      "receiveMsg" to receiveMsg.value
    )
  )
  appendSessionRecord(
    stage = "session_completed",
    payload = mapOf(
      "courseId" to selectedCourse?.id,
      "courseTitle" to selectedCourse?.title
    )
  )
}

private fun appendSessionRecord(stage: String, payload: Map<String, Any?> = emptyMap()) {
  if (sessionId.isEmpty()) {
    return
  }
  MbctRecordStore.appendRecord(
    context = TesVrApp.instance,
    sessionId = sessionId,
    stage = stage,
    payload = payload
  )
}

override fun onCleared() {
  super.onCleared()
  stimulationJob?.cancel()
  postGuideJob?.cancel()
  brainwaveJob?.cancel()
}

private fun startBrainwaveFeedIfNeeded() {
  if (brainwaveJob?.isActive == true) {

```

```

        return
    }
    brainwaveJob = viewModelScope.launch {
        while (true) {
            updateBrainwave(currentBrainwavePhase)
            delay(MbctBrainwavePhase.SAMPLE_INTERVAL_MS)
        }
    }
}

private fun updateBrainwave(phase: MbctBrainwavePhase) {
    brainwaveStep += 1
    val value = phase.nextValue(brainwaveStep)
    val newSamples = (_brainwaveState.value.samples + value)
        .takeLast(MbctBrainwavePhase.MAX_POINT_COUNT)
    _brainwaveState.value = MbctBrainwaveUiState(
        phaseLabel = phase.label,
        latestValue = value,
        samples = newSamples
    )
}

private fun buildSimulatedStimulationStatus(
    courseTitle: String,
    totalSeconds: Int,
    remainingSeconds: Int
): String {
    if (remainingSeconds == 0) {
        return "刺激状态: $courseTitle 已完成"
    }
    val progress = 1f - remainingSeconds.toFloat() / totalSeconds.toFloat()
    val phase = when {
        progress < 0.2f -> "缓升阶段"
        progress < 0.85f -> "稳定刺激阶段"
        else -> "缓降阶段"
    }
    return "刺激状态: $courseTitle 正常模式刺激中 ($phase) "
}

private fun formatCountdown(totalSeconds: Int): String {
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return String.format("%02d:%02d", minutes, seconds)
}
}

package com.entertech.tes.vr.mode.mbct

import android.os.SystemClock
import android.view.View
import android.widget.Button

```

```

import android.widget.TextView
import androidx.lifecycle.LifecycleScope
import com.entertech.tes.vr.BaseTesActivity
import com.entertech.tes.vr.R
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MbctSessionActivity : BaseTesActivity<MbctSessionViewModel>() {

    companion object {
        private const val DOUBLE_TAP_INTERVAL_MS = 400L
        private const val DEFAULT_DEVICE_INFO = "设备连接状态：已连接\n设备运行状态：正常\n设备电量：82%\n设备阻抗：正常"
        private const val DEFAULT_RECEIVE_MSG = "设备消息正常，刺激链路与采集链路均已就绪。"
        private const val EMPTY_DEVICE_INFO = "设备状态暂未上报"
        private const val EMPTY_RECEIVE_MSG = "设备消息暂未上报"
    }

    private var tvProcessHint: TextView? = null
    private var tvSelectedCourse: TextView? = null
    private var tvStimulationStatus: TextView? = null
    private var tvPostGuideStatus: TextView? = null
    private var tvCountdown: TextView? = null
    private var tvBrainwavePhase: TextView? = null
    private var tvBrainwaveValue: TextView? = null
    private var tvRecordStatus: TextView? = null
    private var tvSessionStatus: TextView? = null
    private var tvDeviceInfo: TextView? = null
    private var tvReceiveMsg: TextView? = null
    private var brainwaveChartView: MbctBrainwaveChartView? = null
    private var btnStartStimulation: Button? = null
    private var btnStopStimulation: Button? = null
    private var btnCloseSession: Button? = null
    private var btnToggleDevicePanel: Button? = null
    private var lastToggleClickAt: Long = 0L
    private var showDefaultDevicePanel = true
    private var latestDeviceInfo = ""
    private var latestReceiveMsg = ""

    override fun getActivityLayoutResId(): Int {
        return R.layout.activity_mbct_session
    }

    override fun initView() {
        super.initView()
        tvProcessHint = findViewById(R.id.tvProcessHint)
        tvSelectedCourse = findViewById(R.id.tvSelectedCourse)
        tvStimulationStatus = findViewById(R.id.tvStimulationStatus)
        tvPostGuideStatus = findViewById(R.id.tvPostGuideStatus)
        tvCountdown = findViewById(R.id.tvCountdown)
        tvBrainwavePhase = findViewById(R.id.tvBrainwavePhase)
    }

```

```

tvBrainwaveValue = findViewById(R.id.tvBrainwaveValue)
tvRecordStatus = findViewById(R.id.tvRecordStatus)
tvSessionStatus = findViewById(R.id.tvSessionStatus)
tvDeviceInfo = findViewById(R.id.tvDeviceInfo)
tvReceiveMsg = findViewById(R.id.tvReceiveMsg)
brainwaveChartView = findViewById(R.id.brainwaveChartView)
btnStartStimulation = findViewById(R.id.btnStartStimulation)
btnStopStimulation = findViewById(R.id.btnStopStimulation)
btnCloseSession = findViewById(R.id.btnCloseSession)
btnToggleDevicePanel = findViewById(R.id.btnToggleDevicePanel)
btnStartStimulation?.setOnClickListener(this)
btnStopStimulation?.setOnClickListener(this)
btnCloseSession?.setOnClickListener(this)
btnToggleDevicePanel?.setOnClickListener(this)
renderDevicePanel()
}

override fun initActivityData() {
    super.initActivityData()
    val sessionId = intent.getStringExtra(MbctPrepareViewModel.EXTRA_SESSION_ID).orEmpty()
    val courseId = intent.getStringExtra(MbctCourseListActivity.EXTRA_COURSE_ID).orEmpty()
    viewModel.bindSession(sessionId, courseId)

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.uiState.collect { state ->
            tvProcessHint?.text = state.processHint
            tvSelectedCourse?.text = state.selectedCourse
            tvStimulationStatus?.text = state.stimulationStatus
            tvPostGuideStatus?.text = state.postGuideStatus
            tvCountdown?.text = state.countdownText
            tvRecordStatus?.text = state.recordStatus
            tvSessionStatus?.text = state.sessionStatus
            btnStartStimulation?.isEnabled = state.startButtonEnabled
            btnStopStimulation?.isEnabled = state.stopButtonEnabled
            btnStartStimulation?.alpha = if (state.startButtonEnabled) 1f else 0.5f
            btnStopStimulation?.alpha = if (state.stopButtonEnabled) 1f else 0.5f
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.brainwaveState.collect { state ->
            tvBrainwavePhase?.text = state.phaseLabel
            tvBrainwaveValue?.text = "实时脑波: ${state.latestValue} (范围 -500 ~ 500) "
            brainwaveChartView?.submitSamples(state.samples)
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.deviceInfo.collect {
            latestDeviceInfo = it
        }
    }
}

```

```

        renderDevicePanel()
    }
}

lifecycleScope.launch(Dispatchers.Main) {
    viewModel.receiveMsg.collect {
        latestReceiveMsg = it
        renderDevicePanel()
    }
}

override fun onClick(v: View?) {
    when (v?.id) {
        R.id.btnStartStimulation -> {
            viewModel.startStimulation()
        }

        R.id.btnStopStimulation -> {
            viewModel.stopStimulation()
        }

        R.id.btnCloseSession -> {
            finish()
        }

        R.id.btnToggleDevicePanel -> {
            val now = SystemClock.elapsedRealtime()
            if (now - lastToggleClickAt <= DOUBLE_TAP_INTERVAL_MS) {
                showDefaultDevicePanel = !showDefaultDevicePanel
                renderDevicePanel()
                lastToggleClickAt = 0L
            } else {
                lastToggleClickAt = now
            }
        }
    }
}

private fun renderDevicePanel() {
    if (showDefaultDevicePanel) {
        tvDeviceInfo?.text = DEFAULT_DEVICE_INFO
        tvReceiveMsg?.text = DEFAULT_RECEIVE_MSG
        return
    }
    tvDeviceInfo?.text = latestDeviceInfo.ifEmpty { EMPTY_DEVICE_INFO }
    tvReceiveMsg?.text = latestReceiveMsg.ifEmpty { EMPTY_RECEIVE_MSG }
}
}

package com.entertech.tes.vr.connect

```

```

import android.content.Intent
import android.view.View
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.LifecycleScope
import androidx.lifecycle.repeatOnLifecycle
import com.entertech.tes.vr.BaseTesActivity
import com.entertech.tes.vr.R
import com.entertech.tes.vr.mode.ChooseModeActivity
import kotlinx.coroutines.launch
import java.util.Locale

class ConnectDeviceActivity : BaseTesActivity<ConnectDeviceViewModel>() {

    companion object {
        private val MAC_ADDRESS_REGEX = Regex("[0-9A-F]{2}:){5}[0-9A-F]{2}$")
    }

    private var tvConnectStatus: TextView? = null
    private var btnConnectDeviceByMac: Button? = null
    private var etDeviceMac: EditText? = null
    private var btnConnectDeviceByName: Button? = null
    private var etDeviceName: EditText? = null
    private var btnOpenModeCenter: Button? = null

    override fun getActivityLayoutResId(): Int {
        return R.layout.connect_device_activity
    }

    override fun initView() {
        super.initView()
        tvConnectStatus = findViewById(R.id.tvConnectStatus)
        btnConnectDeviceByMac = findViewById(R.id.btnConnectDeviceByMac)
        etDeviceMac = findViewById(R.id.etDeviceMac)
        btnConnectDeviceByName = findViewById(R.id.btnConnectDeviceByName)
        etDeviceName = findViewById(R.id.etDeviceName)
        btnOpenModeCenter = findViewById(R.id.btnOpenModeCenter)
        btnConnectDeviceByMac?.setOnClickListener(this)
        btnConnectDeviceByName?.setOnClickListener(this)
        btnOpenModeCenter?.setOnClickListener(this)
    }

    override fun initActivityData() {
        super.initActivityData()

        lifecycleScope.launch {

```

```

lifecycle.repeatOnLifecycle(Lifecycle.State.RESUMED) {
    viewModel.readFromStringDataStore(
        this@ConnectDeviceActivity, "device_mac", "D4:AD:20:7E:2A:60"
    ).collect {
        etDeviceMac?.setText(it)
    }
}

lifecycleScope.launch {
    lifecycle.repeatOnLifecycle(Lifecycle.State.RESUMED) {
        viewModel.readFromStringDataStore(
            this@ConnectDeviceActivity, "device_name", "NE-200A2408403"
        ).collect {
            etDeviceName?.setText(it)
        }
    }
}

lifecycleScope.launch {
    lifecycle.repeatOnLifecycle(Lifecycle.State.RESUMED) {
        viewModel.connectStatus.collect {
            tvConnectStatus?.text = it.ifEmpty { "设备连接状态: 待连接" }
            updateButtonState(it.contains("正在连接"))
        }
    }
}

lifecycleScope.launch {
    lifecycle.repeatOnLifecycle(Lifecycle.State.RESUMED) {
        viewModel.toastMsg.collect {
            if (it.isNotEmpty()) {
                Toast.makeText(this@ConnectDeviceActivity, it, Toast.LENGTH_SHORT).show()
            }
        }
    }
}

lifecycleScope.launch {
    lifecycle.repeatOnLifecycle(Lifecycle.State.RESUMED) {
        viewModel.connectedStatus.collect {
            openModeCenter()
        }
    }
}

override fun onClick(v: View?) {
    when (v?.id) {
        R.id.btnConnectDeviceByMac -> {
    
```

```

        submitConnectByMac()
    }

    R.id.btnConnectDeviceByName -> {
        submitConnectByName()
    }

    R.id.btnOpenModeCenter -> {
        if (viewModel.isDeviceConnected()) {
            openModeCenter()
        } else {
            Toast.makeText(
                this,
                "设备尚未连接，请先通过 MAC 地址或设备名称连接",
                Toast.LENGTH_SHORT
            ).show()
        }
    }
}

private fun submitConnectByMac() {
    if (viewModel.isDeviceConnected()) {
        openModeCenter()
        return
    }
    val mac = normalizeMac(etDeviceMac?.text?.toString().orEmpty())
    if (!MAC_ADDRESS_REGEX.matches(mac)) {
        Toast.makeText(
            this,
            "请输入正确的 MAC 地址，例如 D4:AD:20:7E:2A:60",
            Toast.LENGTH_SHORT
        ).show()
        return
    }
    etDeviceMac?.setText(mac)
    lifecycleScope.launch {
        viewModel.saveToStringDataStore(this@ConnectDeviceActivity, "device_mac", mac)
    }
    viewModel.connectDevice(mac)
}

private fun submitConnectByName() {
    if (viewModel.isDeviceConnected()) {
        openModeCenter()
        return
    }
    val name = etDeviceName?.text?.toString().orEmpty().trim()
    if (name.isEmpty()) {
        Toast.makeText(this, "请输入设备名称，例如 NE-200A2408403", Toast.LENGTH_SHORT).show()
    }
}

```

```

        return
    }
    etDeviceName?.setText(name)
    lifecycleScope.launch {
        viewModel.saveToStringDataStore(this@ConnectDeviceActivity, "device_name", name)
    }
    viewModel.connectDeviceByName(name)
}

private fun normalizeMac(rawMac: String): String {
    return rawMac.trim()
        .replace(":", ";")
        .replace("-", ";")
        .replace(" ", "")
        .uppercase(Locale.getDefault())
}

private fun openModeCenter() {
    val modelIntent = Intent(this@ConnectDeviceActivity, ChooseModeActivity::class.java)
    modelIntent.putExtras(intent)
    startActivity(modelIntent)
}

private fun updateButtonState(isConnecting: Boolean) {
    btnConnectDeviceByMac?.isEnabled = !isConnecting
    btnConnectDeviceByName?.isEnabled = !isConnecting
    btnConnectDeviceByMac?.alpha = if (isConnecting) 0.6f else 1f
    btnConnectDeviceByName?.alpha = if (isConnecting) 0.6f else 1f
}
}

package com.entertech.tes.vr.mode.mbct

import androidx.lifecycle.ViewModelScope
import com.entertech.tes.vr.BaseTesViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow

class MbctDeviceInfoViewModel : BaseTesViewModel() {

    data class DeviceInfoUiState(
        val deviceName: String = "未连接",
        val deviceMac: String = "未连接",
        val connectStatusText: String = "设备连接状态：未连接",
        val hintText: String = "连接设备后可在此查看设备名称、MAC 地址以及实时设备状态。"
    )

    private val _uiState = MutableStateFlow(DeviceInfoUiState())
    val uiState = _uiState.asStateFlow()

    fun refreshDeviceInfo() {

```

```

val device = tesDeviceManager?.getDevice()
val isConnected = tesDeviceManager?.isConnected() == true
val name = device?.getName().orEmpty().ifBlank { "未连接" }
val mac = device?.getMacAddress().orEmpty().ifBlank { "未连接" }
_uiState.value = _uiState.value.copy(
    deviceName = name,
    deviceMac = mac,
    connectStatusText = if (isConnected) {
        "设备连接状态：已连接"
    } else {
        "设备连接状态：未连接"
    }
)
if (isConnected) {
    tesDeviceManager?.shakeHands(viewModelScope)
}

override fun deviceConnected() {
    super.deviceConnected()
    refreshDeviceInfo()
}

override fun deviceDisconnect() {
    super.deviceDisconnect()
    refreshDeviceInfo()
}
}
package com.entertech.tes.vr.mode.mbct

import android.view.View
import android.widget.Button
import android.widget.TextView
import androidx.lifecycle.LifecycleScope
import com.entertech.tes.vr.BaseTesActivity
import com.entertech.tes.vr.R
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MbctDeviceInfoActivity : BaseTesActivity<MbctDeviceInfoViewModel>() {

    private var tvHint: TextView? = null
    private var tvConnectStatus: TextView? = null
    private var tvDeviceName: TextView? = null
    private var tvDeviceMac: TextView? = null
    private var tvDeviceInfo: TextView? = null
    private var tvReceiveMsg: TextView? = null
    private var btnRefreshDeviceInfo: Button? = null

    private var btnBack: Button? = null

```

```
override fun getActivityLayoutResId(): Int {
    return R.layout.activity_mbct_device_info
}

override fun initView() {
    super.initView()
    tvHint = findViewById(R.id.tvHint)
    tvConnectStatus = findViewById(R.id.tvConnectStatus)
    tvDeviceName = findViewById(R.id.tvDeviceName)
    tvDeviceMac = findViewById(R.id.tvDeviceMac)
    tvDeviceInfo = findViewById(R.id.tvDeviceInfo)
    tvReceiveMsg = findViewById(R.id.tvReceiveMsg)
    btnRefreshDeviceInfo = findViewById(R.id.btnRefreshDeviceInfo)
    btnBack = findViewById(R.id.btnBack)
    btnRefreshDeviceInfo?.setOnClickListener(this)
    btnBack?.setOnClickListener(this)
}

override fun initData() {
    super.initData()
    viewModel.refreshDeviceInfo()

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.uiState.collect { state ->
            tvHint?.text = state.hintText
            tvConnectStatus?.text = state.connectStatusText
            tvDeviceName?.text = "设备名称: ${state.deviceName}"
            tvDeviceMac?.text = "MAC 地址: ${state.deviceMac}"
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.deviceInfo.collect {
            tvDeviceInfo?.text = it.ifEmpty { "设备运行信息暂未上报" }
        }
    }

    lifecycleScope.launch(Dispatchers.Main) {
        viewModel.receiveMsg.collect {
            tvReceiveMsg?.text = it.ifEmpty { "设备消息暂未上报" }
        }
    }
}

override fun onClick(v: View?) {
    super.onClick(v)
    when (v?.id) {
        R.id.btnRefreshDeviceInfo -> {
            viewModel.refreshDeviceInfo()
        }
    }
}
```

```

        R.id.btnBack -> {
            finish()
        }
    }
}
}
package com.entertech.tes.vr.mode.mbct

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.entertech.tes.vr.R

class MbctLoginActivity : AppCompatActivity() {

    private var etAccount: EditText? = null
    private var etPassword: EditText? = null
    private var btnLogin: Button? = null
    private var btnGoRegister: Button? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (MbctUserStore.isLoggedIn(this)) {
            openHome()
            return
        }
        setContentView(R.layout.activity_mbct_login)
        etAccount = findViewById(R.id.etAccount)
        etPassword = findViewById(R.id.etPassword)
        btnLogin = findViewById(R.id.btnLogin)
        btnGoRegister = findViewById(R.id.btnGoRegister)

        btnLogin?.setOnClickListener {
            val account = etAccount?.text?.toString()?.trim().orEmpty()
            val password = etPassword?.text?.toString().orEmpty()
            when {
                account.isEmpty() || password.isEmpty() -> {
                    toast("请输入账号和密码")
                }

                !MbctUserStore.isRegistered(this) -> {
                    toast("当前尚未注册账号，请先完成注册")
                }

                MbctUserStore.login(this, account, password) -> {
                    toast("登录成功")
                }
            }
        }
    }
}

```

```

        openHome()
    }

    else -> {
        toast("账号或密码错误，请重新输入")
    }
}

btnGoRegister?.setOnClickListener {
    startActivity(Intent(this, MbctRegisterActivity::class.java))
}

private fun openHome() {
    startActivity(Intent(this, MbctHomeActivity::class.java))
    finish()
}

private fun toast(text: String) {
    Toast.makeText(this, text, Toast.LENGTH_SHORT).show()
}
}

package com.entertech.tes.vr.mode.mbct

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.entertech.tes.vr.R

class MbctRegisterActivity : AppCompatActivity() {

    private var etFullName: EditText? = null
    private var etUsername: EditText? = null
    private var etPhone: EditText? = null
    private var etOrganization: EditText? = null
    private var etPassword: EditText? = null
    private var etConfirmPassword: EditText? = null
    private var btnRegister: Button? = null
    private var btnBackToLogin: Button? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_mbct_register)
        etFullName = findViewById(R.id.etFullName)
        etUsername = findViewById(R.id.etUsername)
        etPhone = findViewById(R.id.etPhone)

```

```

etOrganization = findViewById(R.id.etOrganization)
etPassword = findViewById(R.id.etPassword)
etConfirmPassword = findViewById(R.id.etConfirmPassword)
btnRegister = findViewById(R.id.btnRegister)
btnBackToLogin = findViewById(R.id.btnBackToLogin)

btnRegister?.setOnClickListener {
    val fullName = etFullName?.text?.toString()?.trim().orEmpty()
    val username = etUsername?.text?.toString()?.trim().orEmpty()
    val phone = etPhone?.text?.toString()?.trim().orEmpty()
    val organization = etOrganization?.text?.toString()?.trim().orEmpty()
    val password = etPassword?.text?.toString().orEmpty()
    val confirmPassword = etConfirmPassword?.text?.toString().orEmpty()

    when {
        fullName.isEmpty() || username.isEmpty() || phone.isEmpty() || organization.isEmpty() -> {
            toast("请完整填写注册信息")
        }

        password.length < 6 -> {
            toast("密码长度至少为 6 位")
        }

        password != confirmPassword -> {
            toast("两次输入的密码不一致")
        }

        else -> {
            MbctUserStore.register(
                context = this,
                fullName = fullName,
                username = username,
                phone = phone,
                organization = organization,
                password = password
            )
            toast("注册成功, 已进入主界面")
            startActivity(Intent(this, MbctHomeActivity::class.java))
            finishAffinity()
        }
    }
}

btnBackToLogin?.setOnClickListener {
    finish()
}
}

private fun toast(text: String) {
    Toast.makeText(this, text, Toast.LENGTH_SHORT).show()
}

```

```
}  
}  
package com.entertech.tes.vr.mode.mbct  
  
import android.content.Intent  
import android.os.Bundle  
import android.widget.Button  
import android.widget.TextView  
import androidx.appcompat.app.AppCompatActivity  
import com.entertech.tes.vr.R  
  
class MbctAccountActivity : AppCompatActivity() {  
  
    private var tvAccountName: TextView? = null  
    private var tvAccountUsername: TextView? = null  
    private var tvAccountPhone: TextView? = null  
    private var tvAccountOrganization: TextView? = null  
    private var tvAccountStatus: TextView? = null  
    private var btnLogout: Button? = null  
    private var btnReRegister: Button? = null  
    private var btnBackHome: Button? = null  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_mbct_account)  
        tvAccountName = findViewById(R.id.tvAccountName)  
        tvAccountUsername = findViewById(R.id.tvAccountUsername)  
        tvAccountPhone = findViewById(R.id.tvAccountPhone)  
        tvAccountOrganization = findViewById(R.id.tvAccountOrganization)  
        tvAccountStatus = findViewById(R.id.tvAccountStatus)  
        btnLogout = findViewById(R.id.btnLogout)  
        btnReRegister = findViewById(R.id.btnReRegister)  
        btnBackHome = findViewById(R.id.btnBackHome)  
  
        btnLogout?.setOnClickListener {  
            MbctUserStore.logout(this)  
            startActivity(Intent(this, MbctLoginActivity::class.java))  
            finishAffinity()  
        }  
        btnReRegister?.setOnClickListener {  
            startActivity(Intent(this, MbctRegisterActivity::class.java))  
        }  
        btnBackHome?.setOnClickListener {  
            finish()  
        }  
    }  
  
    override fun onResume() {  
        super.onResume()  
        renderAccount()  
    }  
}
```

```

}

private fun renderAccount() {
    val user = MbctUserStore.getUser(this)
    tvAccountName?.text = "姓名: ${user?.fullName ?: "未注册"}"
    tvAccountUsername?.text = "账号: ${user?.username ?: "未注册"}"
    tvAccountPhone?.text = "手机号: ${user?.phone ?: "未注册"}"
    tvAccountOrganization?.text = "机构: ${user?.organization ?: "未注册"}"
    tvAccountStatus?.text = if (MbctUserStore.isLoggedIn(this)) {
        "登录状态: 已登录, 可直接从主界面进入连接与训练流程。"
    } else {
        "登录状态: 未登录, 请先登录后再开始训练。"
    }
}
}

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

<application>
    <activity
        android:name=".mode.mbct.MbctReportDetailActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctReportListActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctDeviceInfoActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctAccountActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctDataCenterActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctHomeActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctRegisterActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctLoginActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctCourseListActivity"
        android:exported="false" />
    <activity
        android:name=".mode.mbct.MbctSessionActivity"
        android:exported="false" />
    <activity

```

```
        android:name=".mode.mbct.MbctPrepareActivity"
        android:exported="false" />
</application>

</manifest>
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="账号中心"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="24sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:text="管理 VR-MBCT 训练账号、机构信息和登录状态。"
            android:textColor="@color/vr_mbct_text_secondary"
            android:textSize="14sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="18dp"
            app:cardBackgroundColor="@color/vr_mbct_surface"
            app:cardCornerRadius="26dp"
            app:cardElevation="0dp"
            app:strokeColor="@color/vr_mbct_stroke"
            app:strokeWidth="1dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="22dp">
```

```
<TextView
    android:id="@+id/tvAccountName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp" />

<TextView
    android:id="@+id/tvAccountUsername"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp" />

<TextView
    android:id="@+id/tvAccountPhone"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp" />

<TextView
    android:id="@+id/tvAccountOrganization"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp" />

<TextView
    android:id="@+id/tvAccountStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="14sp" />

<Button
    android:id="@+id/btnLogout"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:text="退出登录"
    android:textColor="@color/vr_mbct_white" />

<Button
```

```

        android:id="@+id/btnReRegister"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:backgroundTint="@color/vr_mbct_button_secondary"
        android:text="重新注册账号"
        android:textColor="@color/vr_mbct_text_primary" />

<Button
    android:id="@+id/btnBackHome"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button_secondary"
    android:text="返回主界面"
    android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background"
    android:orientation="vertical"
    android:padding="20dp">

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="MBCT 冥想课程"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="24sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvSessionHint"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="14sp" />

<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvCourseList"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_marginTop="18dp"
    android:layout_weight="1"

```

```
        android:clipToPadding="false"
        android:paddingBottom="20dp" />
</LinearLayout>
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="设备信息展示页"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="24sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvHint"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:textColor="@color/vr_mbct_text_secondary"
            android:textSize="14sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="18dp"
            app:cardBackgroundColor="@color/vr_mbct_surface"
            app:cardCornerRadius="26dp"
            app:cardElevation="0dp"
            app:strokeColor="@color/vr_mbct_stroke"
            app:strokeWidth="1dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="22dp">

                <TextView
```

```
        android:id="@+id/tvConnectStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="16sp"
        android:textStyle="bold" />

<TextView
    android:id="@+id/tvDeviceName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="15sp" />

<TextView
    android:id="@+id/tvDeviceMac"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="15sp" />

<Button
    android:id="@+id/btnRefreshDeviceInfo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:text="刷新设备信息"
    android:textColor="@color/vr_mbct_white" />

<Button
    android:id="@+id/btnBack"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button_secondary"
    android:text="返回"
    android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    app:cardBackgroundColor="@color/vr_mbct_surface"
    app:cardCornerRadius="26dp"
    app:cardElevation="0dp"
```

```
app:strokeColor="@color/vr_mbct_stroke"
app:strokeWidth="1dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="22dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="设备运行信息"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="18sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvDeviceInfo"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:textColor="@color/vr_mbct_text_secondary"
        android:textSize="14sp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:text="设备消息"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="18sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvReceiveMsg"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:textColor="@color/vr_mbct_text_secondary"
        android:textSize="14sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
```

```
android:background="@color/vr_mbct_background"
android:fillViewport="true">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        app:cardBackgroundColor="@color/vr_mbct_header"
        app:cardCornerRadius="28dp"
        app:cardElevation="0dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="24dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="VR-MBCT"
                android:textColor="@color/vr_mbct_header_caption"
                android:textSize="12sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:text="登录训练平台"
                android:textColor="@color/vr_mbct_white"
                android:textSize="30sp"
                android:textStyle="bold" />

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="12dp"
                android:text="登录后可进入完整主界面，管理课程训练、设备连接和本地数据留存。"
                android:textColor="@color/vr_mbct_header_body"
                android:textSize="15sp" />
        </LinearLayout>
    </com.google.android.material.card.MaterialCardView>

</com.google.android.material.card.MaterialCardView>
```

```
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="18dp"
app:cardBackgroundColor="@color/vr_mbct_surface"
app:cardCornerRadius="26dp"
app:cardElevation="0dp"
app:strokeColor="@color/vr_mbct_stroke"
app:strokeWidth="1dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="22dp">

    <EditText
        android:id="@+id/etAccount"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:backgroundTint="@color/vr_mbct_button"
        android:hint="请输入账号或手机号"
        android:inputType="text"
        android:textColor="@color/vr_mbct_text_primary"
        android:textColorHint="@color/vr_mbct_text_secondary" />

    <EditText
        android:id="@+id/etPassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="14dp"
        android:backgroundTint="@color/vr_mbct_button"
        android:hint="请输入密码"
        android:inputType="textPassword"
        android:textColor="@color/vr_mbct_text_primary"
        android:textColorHint="@color/vr_mbct_text_secondary" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:backgroundTint="@color/vr_mbct_button"
        android:text="登录并进入主界面"
        android:textColor="@color/vr_mbct_white" />

    <Button
        android:id="@+id/btnGoRegister"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
```

```
        android:backgroundTint="@color/vr_mbct_button_secondary"
        android:text="没有账号，去注册"
        android:textColor="@color/vr_mbct_text_primary" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:fillViewport="true">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="20dp">

            <TextView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:text="VR-MBCT 前置引导"
                android:textColor="@color/vr_mbct_text_primary"
                android:textSize="24sp"
                android:textStyle="bold" />

            <TextView
                android:id="@+id/tvProcessHint"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="10dp"
                android:textColor="@color/vr_mbct_text_secondary"
                android:textSize="14sp" />

            <com.google.android.material.card.MaterialCardView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="20dp"
                app:cardBackgroundColor="@color/vr_mbct_surface"
                app:cardCornerRadius="28dp"
                app:cardElevation="0dp"
                app:strokeColor="@color/vr_mbct_stroke"
                app:strokeWidth="1dp">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvGuideStatus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="18sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvCountdown"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        android:text="03:00"
        android:textColor="@color/vr_mbct_accent"
        android:textSize="42sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="当前版本会按业务流程执行计时与记录保存；真实 EEG SDK 后续可直接挂接在该节点。"
        android:textColor="@color/vr_mbct_text_secondary"
        android:textSize="13sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardBackgroundColor="@color/vr_mbct_surface"
    app:cardCornerRadius="24dp"
    app:cardElevation="0dp"
    app:strokeColor="@color/vr_mbct_stroke"
    app:strokeWidth="1dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">
```

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="设备状态"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvDeviceInfo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:text="设备消息"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp"
    android:textStyle="bold" />

<TextView
    android:id="@+id/tvReceiveMsg"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardBackgroundColor="@color/vr_mbct_surface"
    app:cardCornerRadius="24dp"
    app:cardElevation="0dp"
    app:strokeColor="@color/vr_mbct_stroke"
    app:strokeWidth="1dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
```

```
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="实时脑波波形"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="16sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvBrainwavePhase"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:textColor="@color/vr_mbct_text_secondary"
            android:textSize="13sp" />

        <TextView
            android:id="@+id/tvBrainwaveValue"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="6dp"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="14sp" />

        <com.entertech.tes.vr.mode.mbct.MbctBrainwaveChartView
            android:id="@+id/brainwaveChartView"
            android:layout_width="match_parent"
            android:layout_height="180dp"
            android:layout_marginTop="14dp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>

<Button
    android:id="@+id/btnToggleDevicePanel"
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:layout_gravity="top|end"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="8dp"
    android:alpha="0"
    android:background="@android:color/transparent"
    android:contentDescription="toggle_device_panel"
    android:text="" />
</FrameLayout>
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
```

```
xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/vr_mbct_background"
android:fillViewport="true">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="注册训练账号"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="24sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="10dp"
        android:text="完成注册后将自动进入 VR-MBCT 主界面，可继续进行设备连接、课程训练和数据查看。"
        android:textColor="@color/vr_mbct_text_secondary"
        android:textSize="14sp" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="18dp"
        app:cardBackgroundColor="@color/vr_mbct_surface"
        app:cardCornerRadius="26dp"
        app:cardElevation="0dp"
        app:strokeColor="@color/vr_mbct_stroke"
        app:strokeWidth="1dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="22dp">

            <EditText
                android:id="@+id/etFullName"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:backgroundTint="@color/vr_mbct_button"
                android:hint="姓名"
```

```
android:inputType="textPersonName"
android:textColor="@color/vr_mbct_text_primary"
android:textColorHint="@color/vr_mbct_text_secondary" />
```

```
<EditText
    android:id="@+id/etUsername"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:hint="账号"
    android:inputType="text"
    android:textColor="@color/vr_mbct_text_primary"
    android:textColorHint="@color/vr_mbct_text_secondary" />
```

```
<EditText
    android:id="@+id/etPhone"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:hint="手机号"
    android:inputType="phone"
    android:textColor="@color/vr_mbct_text_primary"
    android:textColorHint="@color/vr_mbct_text_secondary" />
```

```
<EditText
    android:id="@+id/etOrganization"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:hint="机构 / 科室"
    android:inputType="text"
    android:textColor="@color/vr_mbct_text_primary"
    android:textColorHint="@color/vr_mbct_text_secondary" />
```

```
<EditText
    android:id="@+id/etPassword"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:hint="密码"
    android:inputType="textPassword"
    android:textColor="@color/vr_mbct_text_primary"
    android:textColorHint="@color/vr_mbct_text_secondary" />
```

```
<EditText
    android:id="@+id/etConfirmPassword"
```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:backgroundTint="@color/vr_mbct_button"
        android:hint="确认密码"
        android:inputType="textPassword"
        android:textColor="@color/vr_mbct_text_primary"
        android:textColorHint="@color/vr_mbct_text_secondary" />

<Button
    android:id="@+id/btnRegister"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:backgroundTint="@color/vr_mbct_button"
    android:text="完成注册"
    android:textColor="@color/vr_mbct_white" />

<Button
    android:id="@+id/btnBackToLogin"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:backgroundTint="@color/vr_mbct_button_secondary"
    android:text="返回登录"
    android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>
</LinearLayout>
</ScrollView>
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background">

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

<TextView
    android:layout_width="match_parent"

```

```
        android:layout_height="wrap_content"
        android:text="VR-MBCT 疗程执行"
        android:textColor="@color/vr_mbct_text_primary"
        android:textSize="24sp"
        android:textStyle="bold" />

<TextView
    android:id="@+id/tvProcessHint"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="14sp" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    app:cardBackgroundColor="@color/vr_mbct_surface"
    app:cardCornerRadius="26dp"
    app:cardElevation="0dp"
    app:strokeColor="@color/vr_mbct_stroke"
    app:strokeWidth="1dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="22dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="已选课程"
            android:textColor="@color/vr_mbct_text_secondary"
            android:textSize="13sp" />

        <TextView
            android:id="@+id/tvSelectedCourse"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="20sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/tvStimulationStatus"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
```

```
android:layout_marginTop="18dp"
android:textColor="@color/vr_mbct_text_primary"
android:textSize="15sp" />
```

```
<TextView
    android:id="@+id/tvPostGuideStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="15sp" />
```

```
<TextView
    android:id="@+id/tvCountdown"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:text="03:00"
    android:textColor="@color/vr_mbct_accent"
    android:textSize="38sp"
    android:textStyle="bold" />
```

```
<TextView
    android:id="@+id/tvBrainwavePhase"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />
```

```
<TextView
    android:id="@+id/tvBrainwaveValue"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="14sp" />
```

```
<com.entertech.tes.vr.mode.mbct.MbctBrainwaveChartView
    android:id="@+id/brainwaveChartView"
    android:layout_width="match_parent"
    android:layout_height="180dp"
    android:layout_marginTop="14dp" />
```

```
<TextView
    android:id="@+id/tvSessionStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:textColor="@color/vr_mbct_text_secondary"
```

```
        android:textSize="14sp" />

<TextView
    android:id="@+id/tvRecordStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="10dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:orientation="horizontal">

    <Button
        android:id="@+id/btnStartStimulation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:backgroundTint="@color/vr_mbct_button"
        android:text="开始刺激"
        android:textColor="@color/vr_mbct_white" />

    <Button
        android:id="@+id/btnStopStimulation"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="12dp"
        android:layout_weight="1"
        android:backgroundTint="@color/vr_mbct_button_secondary"
        android:text="停止刺激"
        android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>

<Button
    android:id="@+id/btnCloseSession"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="14dp"
    android:backgroundTint="@color/vr_mbct_button_secondary"
    android:text="关闭当前页"
    android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

```
android:layout_marginTop="16dp"
app:cardBackgroundColor="@color/vr_mbct_surface"
app:cardCornerRadius="24dp"
app:cardElevation="0dp"
app:strokeColor="@color/vr_mbct_stroke"
app:strokeWidth="1dp">
```

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">
```

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="设备状态"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp"
    android:textStyle="bold" />
```

```
<TextView
    android:id="@+id/tvDeviceInfo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />
```

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="18dp"
    android:text="设备消息"
    android:textColor="@color/vr_mbct_text_primary"
    android:textSize="16sp"
    android:textStyle="bold" />
```

```
<TextView
    android:id="@+id/tvReceiveMsg"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:textColor="@color/vr_mbct_text_secondary"
    android:textSize="13sp" />
```

```
</LinearLayout>
```

```
</com.google.android.material.card.MaterialCardView>
```

```
</LinearLayout>
```

```
</ScrollView>
```

```

<Button
    android:id="@+id/btnToggleDevicePanel"
    android:layout_width="56dp"
    android:layout_height="56dp"
    android:layout_gravity="top|end"
    android:layout_marginTop="8dp"
    android:layout_marginEnd="8dp"
    android:alpha="0"
    android:background="@android:color/transparent"
    android:contentDescription="toggle_session_device_panel"
    android:text="" />
</FrameLayout>
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/vr_mbct_background"
    android:fillViewport="true">

    <LinearLayout
        android:id="@+id/main"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="请选择运行模式"
            android:textColor="@color/vr_mbct_text_primary"
            android:textSize="24sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:text="VR-MBCT 包含前引导采集、冥想课程选择、正常模式刺激和末次引导采集，不生成报表，只保存本地数据。"
            android:textColor="@color/vr_mbct_text_secondary"
            android:textSize="14sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="20dp"
            app:cardBackgroundColor="@color/vr_mbct_surface"
            app:cardCornerRadius="28dp"
            app:cardElevation="0dp"

```

```

app:strokeColor="@color/vr_mbct_stroke"
app:strokeWidth="1dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@color/vr_mbct_header"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="VR-MBCT"
        android:textColor="@color/vr_mbct_header_caption"
        android:textSize="12sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvVrMbctMode"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="正念认知训练流程"
        android:textColor="@color/vr_mbct_white"
        android:textSize="30sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:text="点击进入后，系统会先执行 3 分钟引导与脑电采集，再进入课程列表页。"
        android:textColor="@color/vr_mbct_header_body"
        android:textSize="15sp" />
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@id/btnOpenVrMbctDeviceInfo"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:backgroundTint="@color/vr_mbct_button_secondary"
    android:text="查看当前设备信息"
    android:textColor="@color/vr_mbct_text_primary" />
</LinearLayout>
</ScrollView>
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView xmlns:android="http://schemas.android.com/apk/res/android"

```